Pipes in the Desert
Analysis Model - Final Version

E5 - TEAM A

Supervisor:
Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

March 14th, 2026

4. Analysis model – version 2

4.0. Corrections

4.0.1 Functional Requirements (2.3)

ID	Description	Check	Priority	Source	Use-case
FR-16	A Player shall be able to move only to an adjacent connected element that is limited to pipes and pumps.	Non-adjacent movement is rejected.	Must have	Group decision based on game logic	Move Player
FR - 41	The original problem definition states that the winning team is the one collecting the most water but does not explicitly specify when the game ends. In our interpretation, the game session runs for a predefined duration controlled by a timer. When the timer expires, the system compares the total water delivered to cisterns with the total water lost to the desert and determines the winning team.		Should have	Group decision	End Game / Declare Winner

4.0.2 Use-Case Descriptions

Move Player

Use-case name	Move Player
Actors	Player → Plumber (primary), Saboteur (primary)
Related FRs	FR-15, FR-16, FR-17, FR-18, FR-19, FR-20
Preconditions	It is the player's turn. The game session is active.

Dialog / Scenario	1. Player selects an adjacent connected element (only pipes and pumps) as the destination (FR-16). 2. System rejects movement outside the network (FR-15). 3. System rejects movement to a pipe already occupied by another player (FR-17). 4. If the destination is a pump, multiple players may occupy it simultaneously (FR-18). 5. System enforces turn-based play; only the current player can act (FR-19). 6. Each player is limited to one action per turn — moving and picking up cannot be combined (FR-20). 7. Player movement is validated according to adjacency and occupancy rules; if valid, the player's position is updated. The turn then passes to the next player.

4.1 Object catalog

4.1.1 System

The **System** object represents one complete game session. It is responsible for initializing the session, coordinating turns, tracking whether the game is running, and ending the session when the timer expires. It also connects the main domain objects such as the teams, players, pipe network, timer, and scoreboard.

Other objects interact with the System to start play, advance turns, query the current phase of the session, and trigger the end-game procedure. The System does not perform every domain action directly, but it coordinates the objects that do.

4.1.2 Player

A **Player** represents a human-controlled participant moving on the pipe system. A player has a current position and can perform turn-based actions according to the rules of the game. Player is a common concept shared by both Plumber and Saboteur.

Other objects use Player to determine whose turn it is, where the player is standing, and whether an action is allowed from that location. Player itself provides the basic movement-related responsibility, while role-specific actions belong to its specialized forms.

4.1.3 Plumber

A **Plumber** is a specialized Player whose goal is to maximize the amount of water delivered to cisterns. A Plumber can repair broken pumps, repair punctured pipes, change pump direction, connect free pipe ends, and insert new pumps into the network.

Other domain objects use Plumber when an action requires plumber-specific authorization. The class is responsible only for actions that belong uniquely to the plumber role.

4.1.4 Saboteur

A **Saboteur** is a specialized Player whose goal is to maximize water leakage from the network. A Saboteur can change pump direction and puncture pipes in order to reduce successful water delivery.

Other domain objects use Saboteur when verifying sabotage-specific actions. Like Plumber, it extends the common behavior of Player with its own role-specific responsibilities.

4.1.5 PipeNetwork

The **PipeNetwork** represents the full structure of connected pipes and active elements. It stores the current topology of the system and is responsible for managing connections between network elements.

Other objects query the PipeNetwork to validate movement, attach new elements, split pipes, and recalculate structural relationships after a modification.

4.1.6 NetworkElement

A **NetworkElement** is a common abstraction for all objects that are part of the physical water network. It provides the shared concept of connectivity between pipes and active elements such as springs, cisterns, and pumps.

Other objects use NetworkElement when movement, connection, or adjacency must work uniformly across different concrete network objects.

4.1.7 Pipe

A **Pipe** is a network element that transports water between other elements. A pipe may be intact or punctured, and it may also have a free end. If water reaches a free end or a punctured pipe, the water leaks to the desert.

Other objects use Pipe for movement, connection changes, leakage handling, and inserting new pumps by splitting an existing pipe.

4.1.8 Pump

A **Pump** is an active network element that routes water from one selected input pipe to one selected output pipe. It may connect to multiple pipes, but only one input and one output can be active at a time. A pump can malfunction and then stops transferring water until repaired. It also has a limited connection capacity.

Other objects use Pump for routing changes, water transfer checks, breakdown events, repair actions, and connection validation.

4.1.9 Spring

A **Spring** is a source element from which water enters the network. It is responsible for providing water into the connected system during game progression.

Other objects use Spring as the origin of water flow when the network state is evaluated.

4.1.10 Cistern

A **Cistern** is a destination element that receives water delivered through the network. Water that reaches a cistern contributes to the Plumber team's score. The cistern is also the place where new pipes and pumps are generated according to the game rules.

Other objects use Cistern for score updates and for component generation events.

4.1.11 ScoreBoard

The **ScoreBoard** stores and updates the two team scores during the game. It records the amount of water delivered to cisterns for the Plumber team and the amount of leaked water for the Saboteur team. It also provides the final score comparison when the game ends.

Other objects use ScoreBoard whenever water delivery or leakage is recalculated and when the winner must be declared.

4.1.12 GameTimer

The **GameTimer** represents the time limit of the game session. It counts down from the configured duration and triggers the end-game procedure when the remaining time reaches zero.

Other objects use GameTimer to query the remaining time and determine whether the session should still accept gameplay actions.

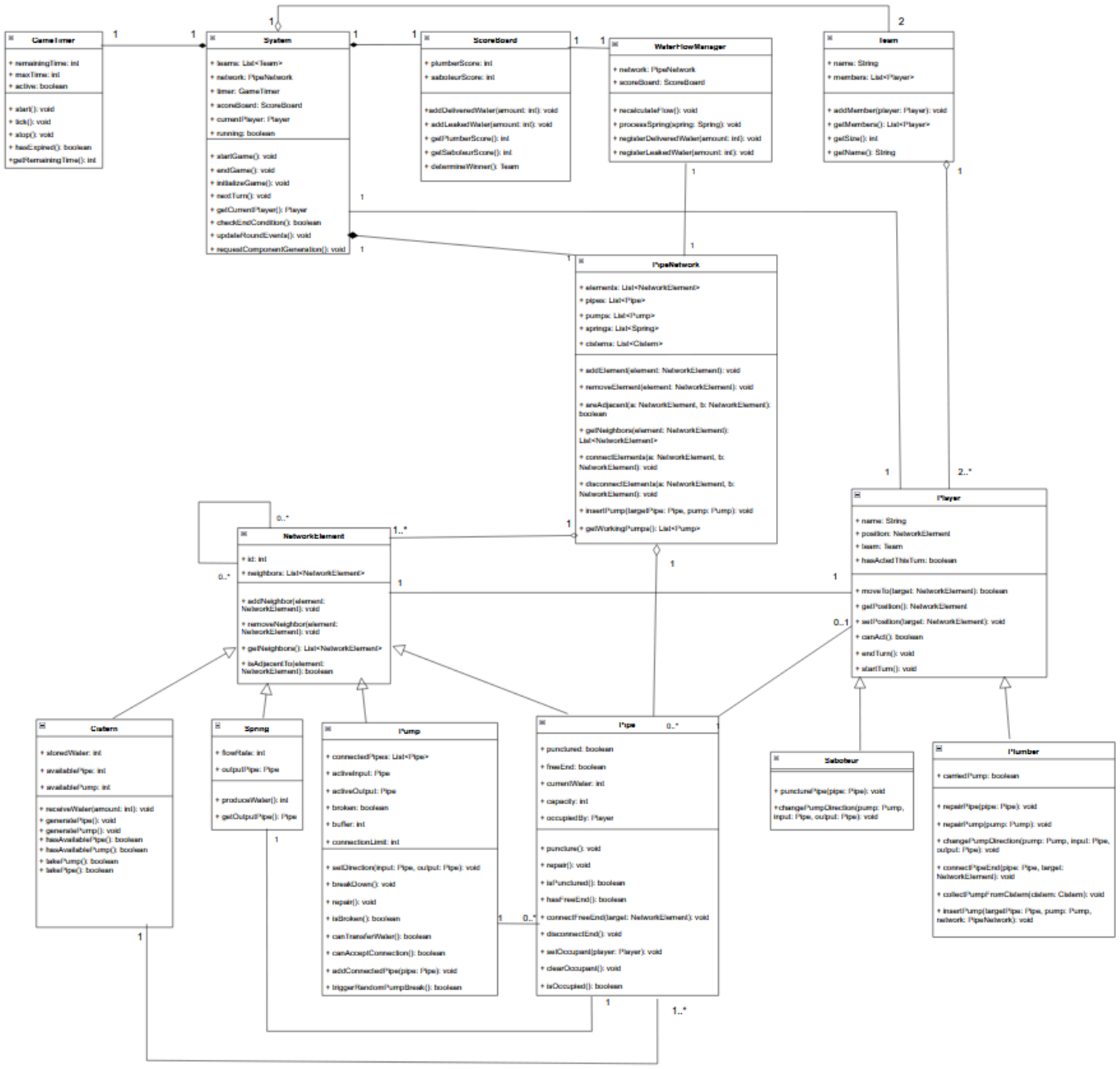
4.1.13 WaterFlowManager

The **WaterFlowManager** is responsible for evaluating how water moves through the current network configuration. It determines whether water is successfully delivered to cisterns, blocked by broken pumps, or leaked through punctured pipes or free pipe ends.

Other objects rely on WaterFlowManager after any structural or state change, such as repairing a pump, changing pump direction, inserting a pump, or puncturing a pipe.

4.2 Static structure diagrams

The model follows a layered object-oriented structure. At the coordination level, System manages the overall game session together with GameTimer, ScoreBoard, and Team. At the domain-structure level, PipeNetwork and NetworkElement describe the physical topology of the game world. At the actor level, Player generalizes the two user roles, Plumber and Saboteur. Finally, WaterFlowManager encapsulates water flow calculation separately from session control. This separation of concerns improves readability and supports a clearer interpretation of the relationships between players and the connected pipe network elements.



4.3 Class description

4.3.1 Cistern

Responsibility

The Cistern class represents a destination node in the pipe network. It stores delivered water and periodically makes new pipes and pumps available according to the game rules. It is also the location where plumbers can collect newly generated pumps, and in our model it is the origin point of newly generated pipes.

Superclasses
NetworkElement

Interfaces
None

Attributes

- storedWater: int — amount of water delivered to this cistern
- availablePipes: int — number of generated pipes currently available
- availablePumps: int — number of generated pumps currently available

Methods

- receiveWater(amount: int): void — adds delivered water to the cistern
- generatePipe(): void — increases the number of available pipes
- generatePump(): void — increases the number of available pumps
- hasAvailablePipe(): boolean — returns whether a pipe is available
- hasAvailablePump(): boolean — returns whether a pump is available
- takePipe(): boolean — removes one available pipe if present
- takePump(): boolean — removes one available pump if present

4.3.2 GameTimer

Responsibility
The GameTimer class manages the time limit of the game session. It counts down from the configured duration and indicates when the game must end. To get rid of ambiguity, our group has decided that the game should have limited duration. So this class is necessary for the end-game logic.

Superclasses
None

Interfaces
None

Attributes

- remainingTime: int — remaining game time
- maxTime: int — initial configured game duration
- active: boolean — indicates whether the timer is currently running

Methods

- start(): void — starts the timer
- tick(): void — decreases remaining time
- stop(): void — stops the timer
- hasExpired(): boolean — returns true if time reached zero
- getRemainingTime(): int — returns remaining tim

4.3.3 NetworkElement

Responsibility
The NetworkElement class represents the common properties of all physical elements that belong to the pipe system. It provides the shared basis for connectivity, position-based interaction, and adjacency inside the network. Pipes, pumps, springs, and cisterns are all treated as network elements in the game world.

Superclasses
None

Interfaces
None

Attributes

- id: int — unique identifier of the element
- neighbors: List<NetworkElement> — directly connected neighboring elements

Methods

- addNeighbor(element: NetworkElement): void — connects another network element
- removeNeighbor(element: NetworkElement): void — removes a connection
- getNeighbors(): List<NetworkElement> — returns connected neighboring elements
- isAdjacentTo(element: NetworkElement): boolean — checks adjacency

4.3.4 Pipe

Responsibility
The Pipe class represents a water-carrying segment in the network. A pipe may be intact or punctured, and it may also have a free end. Water leaks to the desert when the pipe is punctured or when water reaches a free end. Only one player may occupy a pipe at a time. Pipes can also be split when a plumber inserts a new pump.

Superclasses
NetworkElement

Interfaces
None

Attributes

- punctured: boolean — indicates whether the pipe is leaking
- freeEnd: boolean — indicates whether one end is not connected
- currentWater: int — amount of water currently in the pipe
- capacity: int — maximum amount of water the pipe can hold or transfer
- occupiedBy: Player — player currently standing on the pipe, if any

Methods

- puncture(): void — changes the pipe to leaking state
- repair(): void — restores the pipe to intact state
- isPunctured(): boolean — returns whether the pipe is leaking

- hasFreeEnd(): boolean — returns whether the pipe has an unconnected end
- connectFreeEnd(target: NetworkElement): void — connects the free end to another element
- disconnectEnd(): void — disconnects one end of the pipe
- setOccupant(player: Player): void — sets the player occupying the pipe
- clearOccupant(): void — removes the current occupant
- isOccupied(): boolean — returns whether the pipe is occupied

4.3.5 PipeNetwork

Responsibility
The PipeNetwork class stores the full topology of the pipe system. It manages the collection of pipes and active elements, validates adjacency and connections, and supports structural modifications such as connecting a free pipe end or inserting a pump into a pipe.

Superclasses
None

Interfaces
None

Attributes

- elements: List<NetworkElement> — all elements in the network
- pipes: List<Pipe> — all pipes in the network
- pumps: List<Pump> — all pumps in the network
- springs: List<Spring> — all springs in the network
- cisterns: List<Cistern> — all cisterns in the network

Methods

- addElement(element: NetworkElement): void — adds an element to the network
- removeElement(element: NetworkElement): void — removes an element from the network
- areAdjacent(a: NetworkElement, b: NetworkElement): boolean — checks whether two elements are connected
- getNeighbors(element: NetworkElement): List<NetworkElement> — returns neighboring elements
- connectElements(a: NetworkElement, b: NetworkElement): void — creates a connection between two elements
- disconnectElements(a: NetworkElement, b: NetworkElement): void — removes a connection
- insertPump(targetPipe: Pipe, pump: Pump): void — splits a pipe and places a pump between the two resulting segments
- getWorkingPumps(): List<Pump> — returns the pumps currently in working state

4.3.6 Player

Responsibility
The Player class represents a human-controlled participant in the game. It stores the player's

identity, current location, and turn-related status. Shared behavior such as movement and turn completion belongs here, while role-specific actions belong to Plumber and Saboteur.

Superclasses
None

Interfaces
None

Attributes

- name: String — player name
- position: NetworkElement — current position in the network
- team: Team — team to which the player belongs
- hasActedThisTurn: boolean — indicates whether the player already performed an action this turn

Methods

- moveTo(target: NetworkElement): boolean — attempts to move the player to a connected pipe or pump.
- getPosition(): NetworkElement — returns current position
- setPosition(target: NetworkElement): void — sets the player's current position to a valid pipe or pump.
- canAct(): boolean — returns whether the player may still act this turn
- endTurn(): void — marks the player's turn as finished
- startTurn(): void — resets per-turn action status

4.3.7 Plumber

Responsibility
The Plumber class represents a player whose objective is to maximize delivered water. A plumber can repair broken pumps, repair punctured pipes, change pump direction, connect the free end of a generated pipe, collect generated pumps from cisterns, and insert a new pump into an existing pipe. These actions are explicitly supported by the problem definition.

Superclasses
Player

Interfaces
None

Attributes

- carriedPump: boolean — indicates whether the plumber currently carries a collected pump

Methods

- repairPipe(pipe: Pipe): void — repairs a punctured pipe
- repairPump(pump: Pump): void — repairs a broken pump

- changePumpDirection(pump: Pump, input: Pipe, output: Pipe): void — changes the active routing of a pump
- connectPipeEnd(pipe: Pipe, target: NetworkElement): void — performs the plumber action of connecting a pipe's free end to an adjacent element by invoking the pipe's connection behavior.
- collectPumpFromCistern(cistern: Cistern): void — collects a generated pump from an adjacent cistern while standing on a connected pipe or pump.
- insertPump(targetPipe: Pipe, pump: Pump, network: PipeNetwork): void — inserts a pump into a pipe by splitting it

4.3.8 Pump

Responsibility
The Pump class represents an active routing element in the network. It can have multiple connected pipes, but only one active input pipe and one active output pipe at a time. It may malfunction, and when broken it cannot transfer water. It also has a connection limit and an internal water buffer, both of which come directly from the problem definition and glossary.

Superclasses
NetworkElement

Interfaces
None

Attributes

- connectedPipes: List<Pipe> — pipes attached to this pump
- activeInput: Pipe — currently selected input pipe
- activeOutput: Pipe — currently selected output pipe
- broken: boolean — indicates whether the pump is malfunctioning
- buffer: int — internal temporary water storage
- connectionLimit: int — maximum number of connectable pipes

Methods

- setDirection(input: Pipe, output: Pipe): void — selects active input and output pipes
- breakDown(): void — changes the pump to broken state
- repair(): void — restores the pump to working state
- isBroken(): boolean — returns whether the pump is malfunctioning
- canTransferWater(): boolean — checks whether water may pass through the pump
- canAcceptConnection(): boolean — checks whether another pipe may be connected
- addConnectedPipe(pipe: Pipe): void — attaches a pipe to the pump

4.3.9 Saboteur

Responsibility
The Saboteur class represents a player whose objective is to maximize water leakage. A saboteur can puncture pipes and can also change pump direction to divert flow away from cisterns. These are the role-specific sabotage actions described in the problem definition and use cases.

Superclasses
Player

Interfaces
None

Attributes
No additional public attributes are necessary in the first version.

Methods

- puncturePipe(pipe: Pipe): void — punctures an intact pipe
- changePumpDirection(pump: Pump, input: Pipe, output: Pipe): void — changes the routing of a working pump

4.3.10 ScoreBoard

Responsibility
The ScoreBoard class stores the scores of the two teams during the game. One score represents water delivered to cisterns and the other represents water leaked from the network. It is also responsible for comparing the final values and determining the winner at game end.

Superclasses
None

Interfaces
None

Attributes

- plumberScore: int — amount of water delivered to cisterns
- saboteurScore: int — amount of leaked water

Methods

- addDeliveredWater(amount: int): void — increases the plumber score
- addLeakedWater(amount: int): void — increases the saboteur score
- getPlumberScore(): int — returns plumber score
- getSaboteurScore(): int — returns saboteur score
- determineWinner(): Team — returns the winning team based on final comparison

4.3.11 Spring

Responsibility
The Spring class represents a source node from which water enters the network. It produces water that then flows through pipes and pumps toward cisterns. Springs are part of the initial network and are essential for the water-flow logic of the game.

Superclasses
NetworkElement

Interfaces
None

Attributes

- flowRate: int — amount of water produced during one simulation step
- outputPipe: Pipe — pipe connected to the spring

Methods

- produceWater(): int — generates water for the current simulation step
- getOutputPipe(): Pipe — returns connected output pipe

4.3.12 System

Responsibility
The System class coordinates one full game session. It initializes the game, manages turn order, starts and stops the timer, triggers periodic system events, and ends the game when the timer expires. It does not replace the responsibilities of all other classes, but it coordinates them.

Superclasses
None

Interfaces
None

Attributes

- teams: List<Team> — the two competing teams
- network: PipeNetwork — the active network
- timer: GameTimer — session timer
- scoreBoard: ScoreBoard — current scores
- currentPlayer: Player — player whose turn is active
- running: boolean — indicates whether the game session is active
-

Methods

- startGame(): void — starts a new game session
- endGame(): void — terminates the session and triggers winner declaration
- initializeGame(): void — creates the initial network and prepares players
- nextTurn(): void — advances control to the next player
- getCurrentPlayer(): Player — returns the active player
- checkEndCondition(): boolean — checks whether the timer expired
- updateRoundEvents(): void — signals a round update so elements (pumps, cisterns) can trigger their internal events.
- requestComponentGeneration(): void — asks cisterns to generate new pipes or pumps when generation conditions are met.

4.3.13 Team

Responsibility
The Team class represents one side in the game: the plumber team or the saboteur team. It groups players sharing the same goal and makes it possible to express the winner and score comparison at team level instead of only at player level. Since the problem definition explicitly describes two teams, this class is justified and useful.

Superclasses
None

Interfaces
None

Attributes

- name: String — team name
- members: List<Player> — players belonging to the team

Methods

- addMember(player: Player): void — adds a player to the team
- getMembers(): List<Player> — returns team members
- getSize(): int — returns number of players
- getName(): String — returns team name

4.3.14 WaterFlowManager

Responsibility
The WaterFlowManager class evaluates how water moves through the current network configuration. It determines whether water is delivered to cisterns, blocked by broken pumps, leaked through punctured pipes, or lost through free pipe ends. This is a good responsibility separation because many of your use cases explicitly say that the system recalculates water flow after a change in the network.

Superclasses
None

Interfaces
None

Attributes

- network: PipeNetwork — the network being evaluated
- scoreBoard: ScoreBoard — score holder updated according to flow result

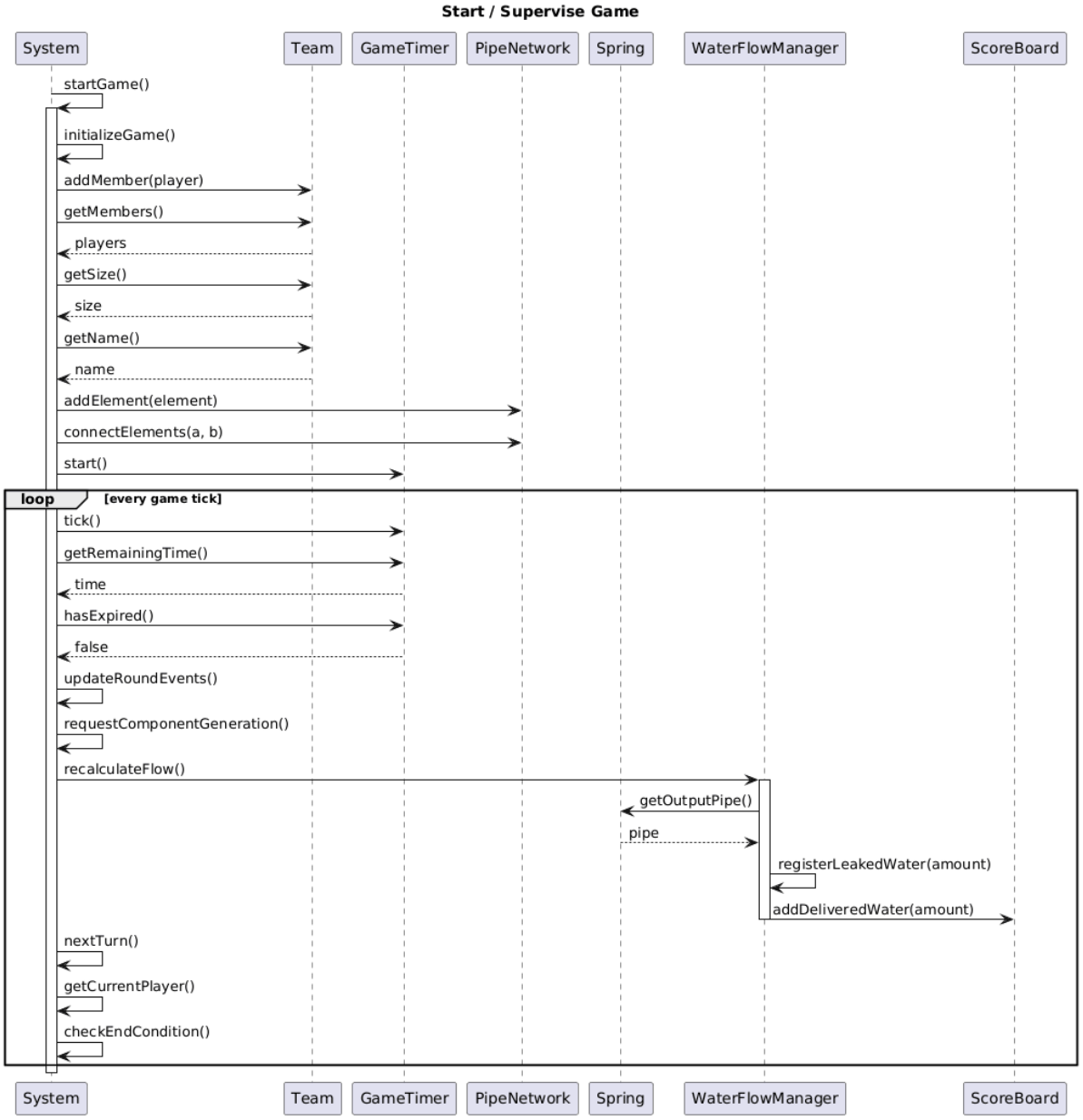
Methods

- recalculateFlow(): void — recalculates water movement in the network
- processSpring(spring: Spring): void — starts flow processing from a spring
- registerDeliveredWater(amount: int): void — records delivered water
- registerLeakedWater(amount: int): void — records leaked water

4.4 Sequence diagrams

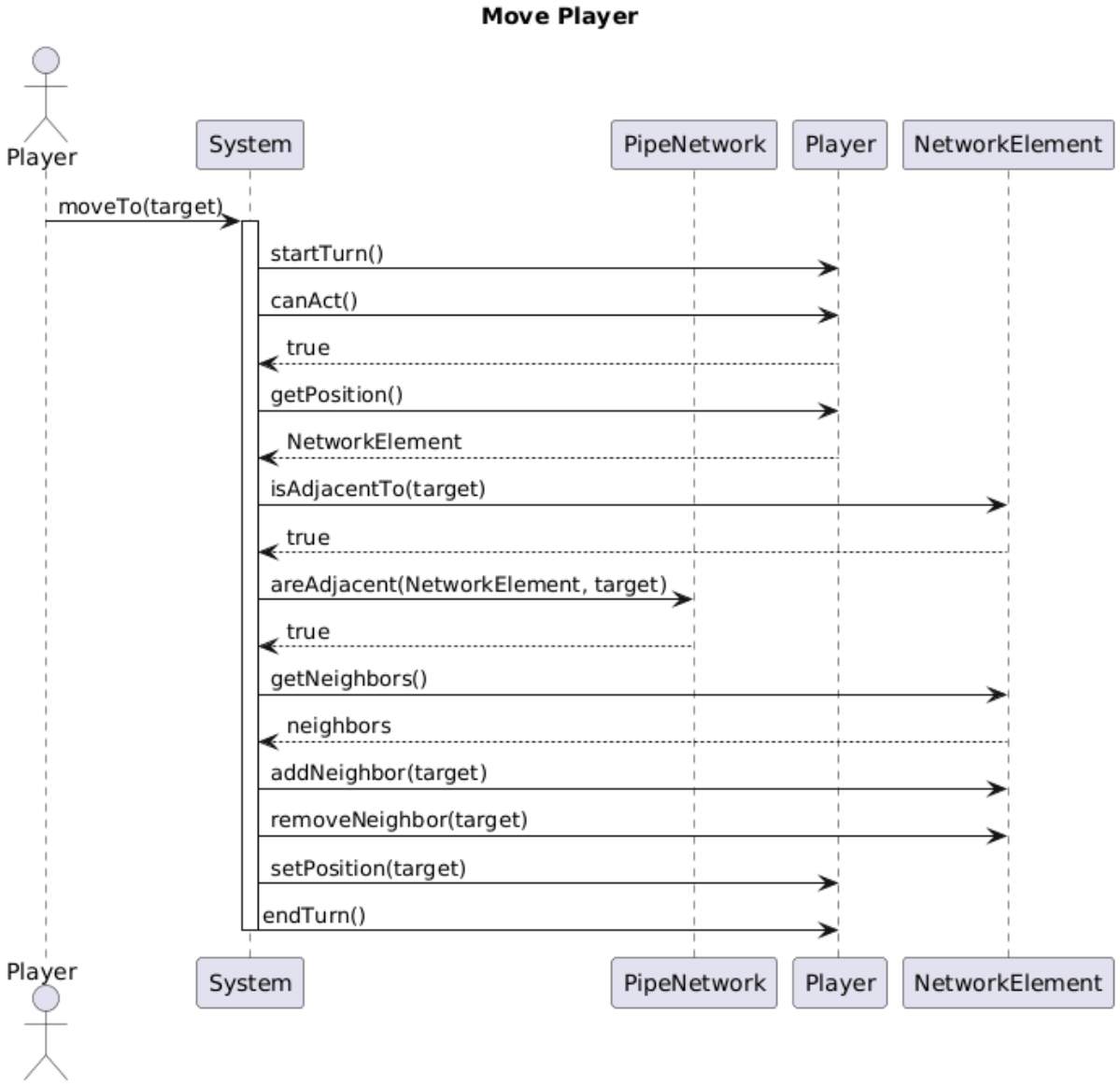
4.4.1 Start / Supervise Game

The System initializes the game by setting up teams, generating the initial pipe network, and starting the timer. It then loops through game ticks managing round events, component generation, water flow, and player turns. The game continues until the timer expires, at which point the end game procedure is triggered.



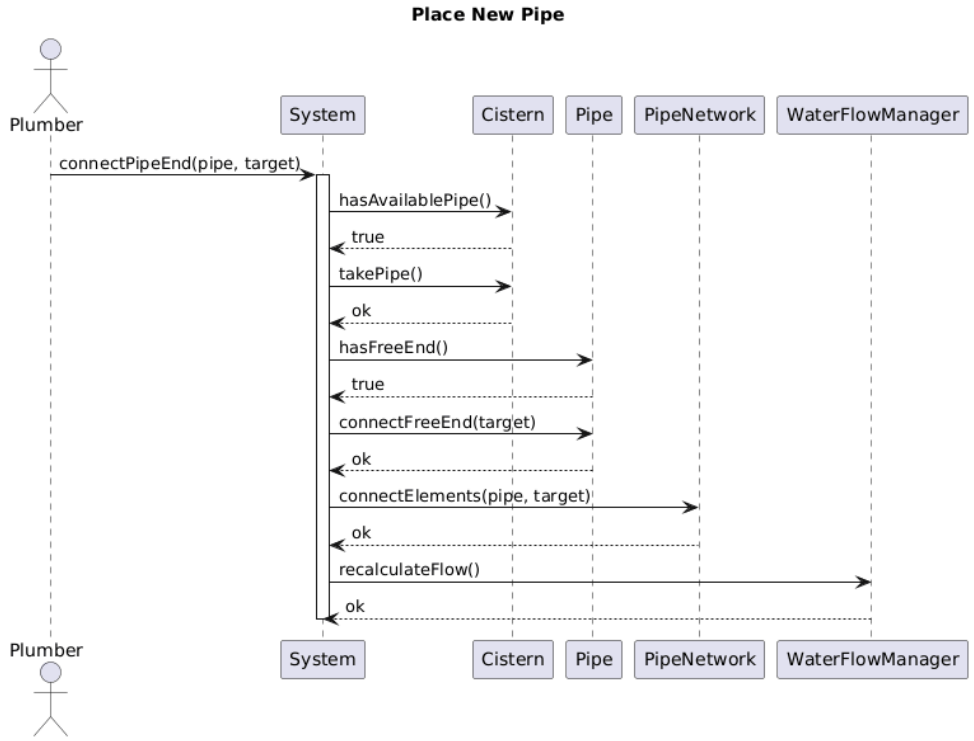
4.4.2 Move Player

A player requests to move to an adjacent network element. The System validates the player's position, checks adjacency, and updates the player's position if the move is valid. Note that according to the problem definition, players can only move onto pipes and pumps; cisterns and springs are not valid destinations.


Move Player
Player
System
PipeNetwork
Player
NetworkElement
moveTo(target)
startTurn()
canAct()
true
getPosition()
NetworkElement
isAdjacentTo(target)
true
areAdjacent(NetworkElement, target)
true
getNeighbors()
neighbors
addNeighbor(target)
removeNeighbor(target)
setPosition(target)
endTurn()
Player
System
PipeNetwork
Player
NetworkElement

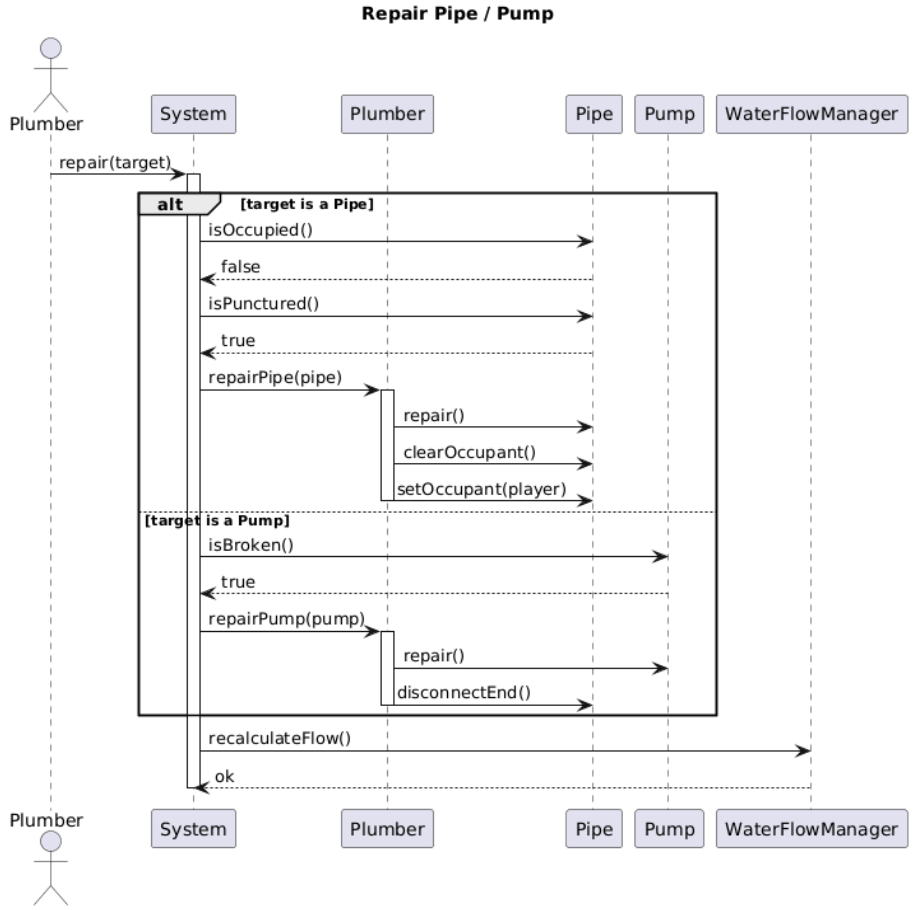
4.4.3 Place New Pipe

A Plumber selects an available pipe at a cistern and connects its free end to a target network element. The System validates the connection and recalculates water flow if the placement is valid.



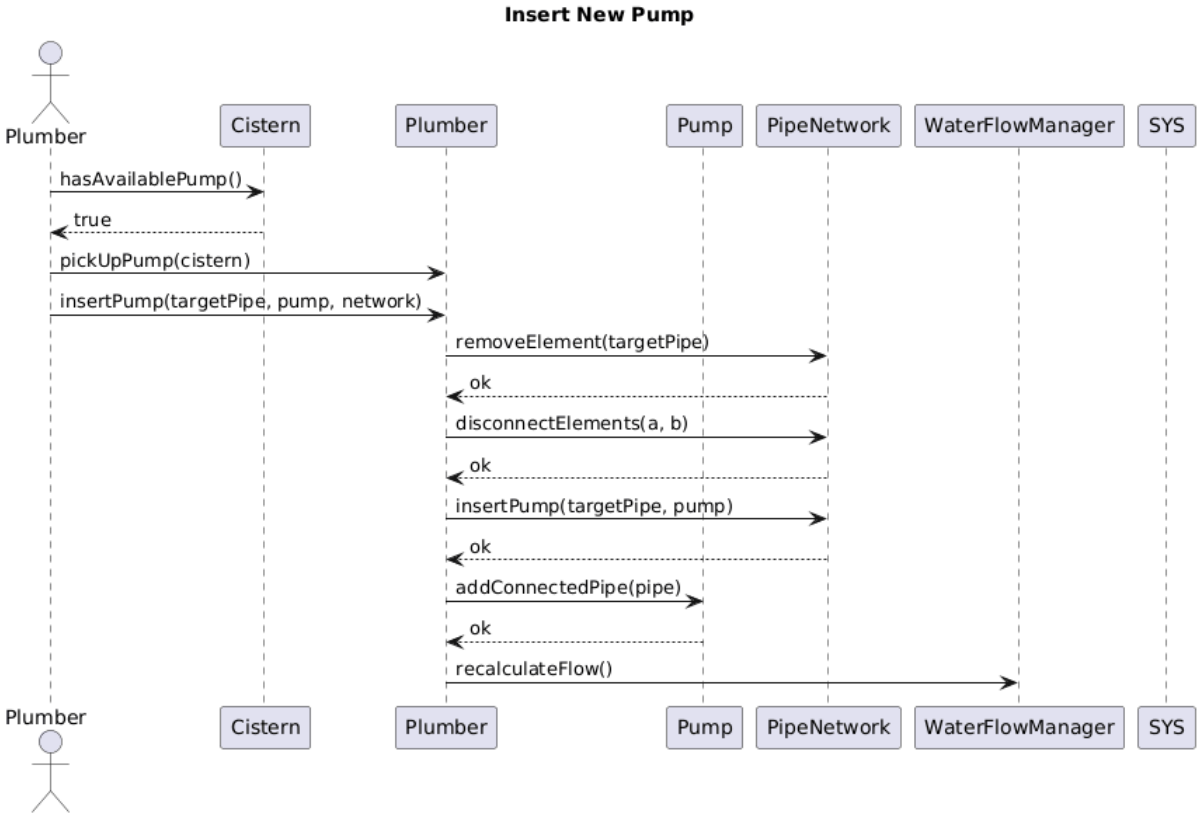
4.4.4 Repair Pipe / Pump

A Plumber repairs either a punctured pipe or a broken pump depending on the target element they are standing on. Water flow is recalculated after the repair is complete.



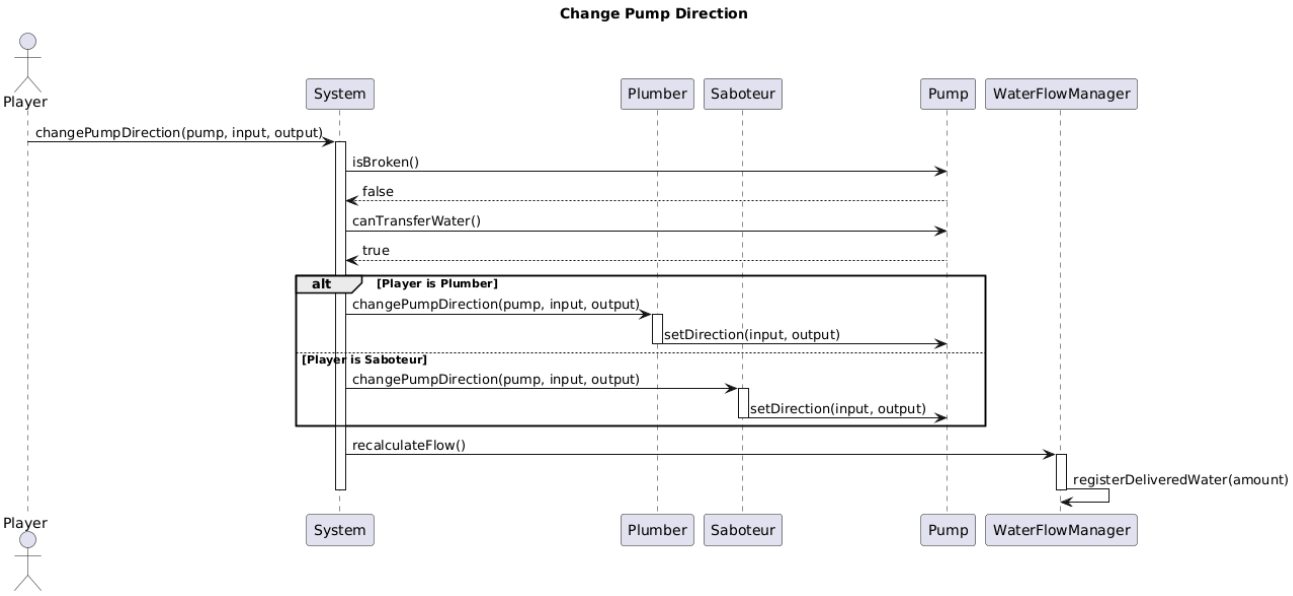
4.4.5 Insert New Pump

A Plumber collects a pump from a cistern and inserts it into an existing pipe by splitting it into two segments. The network is updated and water flow is recalculated.



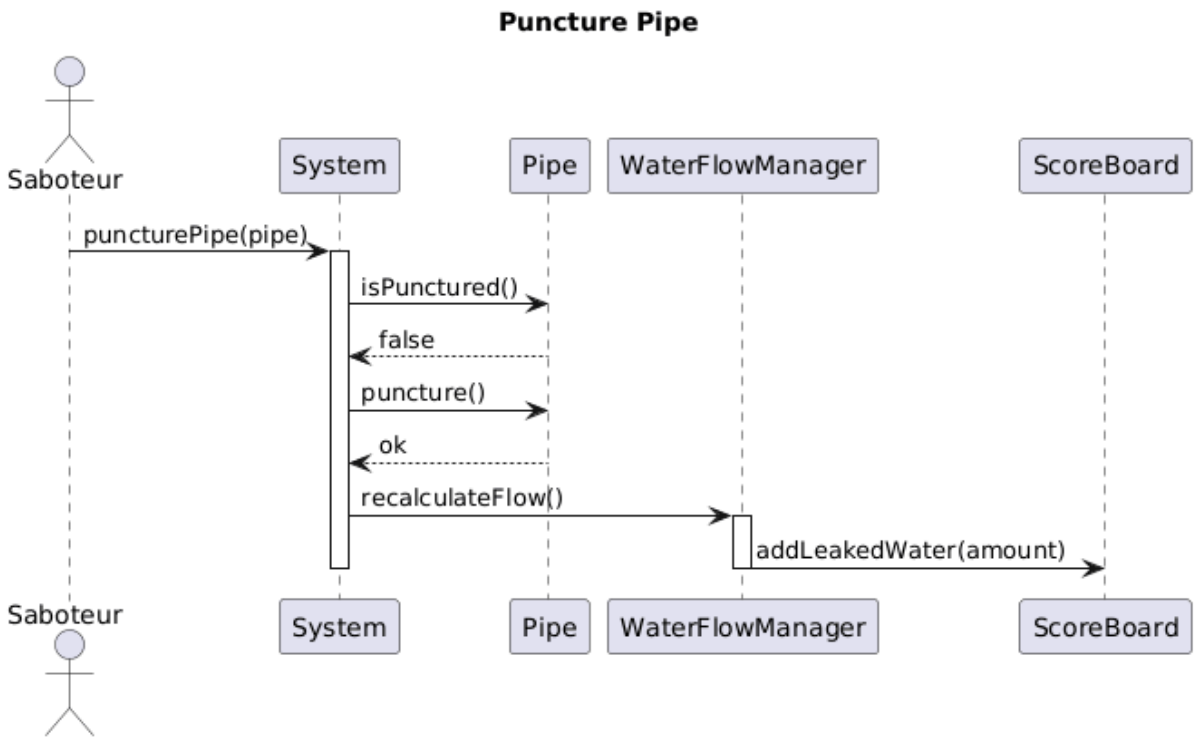
4.4.6 Change Pump Direction

Either a Plumber or Saboteur changes the active input/output routing of a pump they are standing on. The System verifies the pump is working before applying the new direction and recalculating flow.



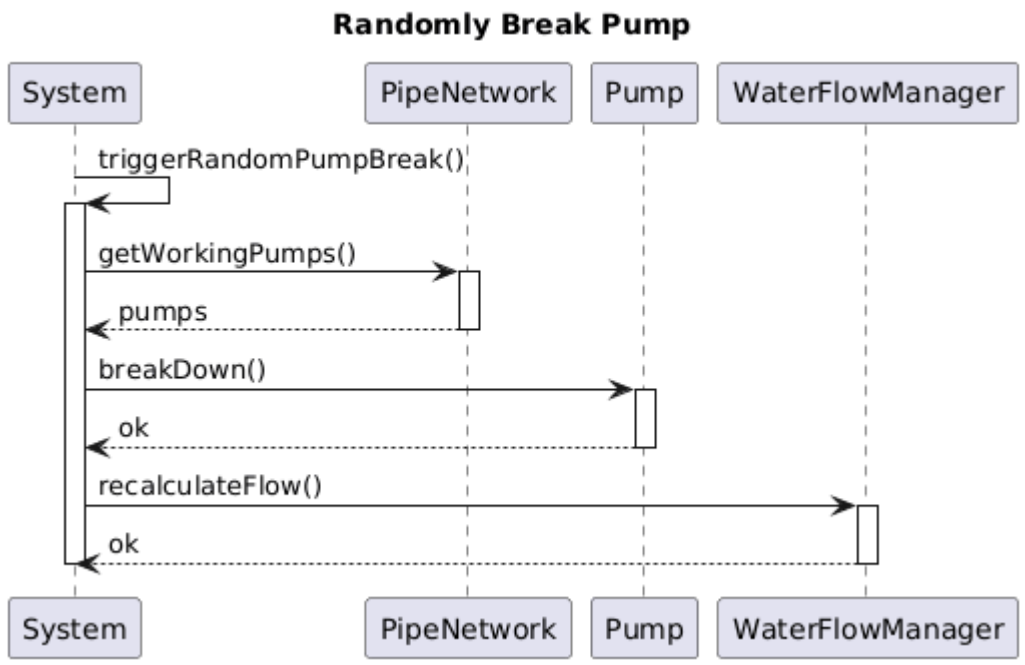
4.4.7 Puncture Pipe

A Saboteur punctures an intact pipe they are standing on, causing water to leak to the desert. The System updates the pipe state and increases the saboteur score accordingly.



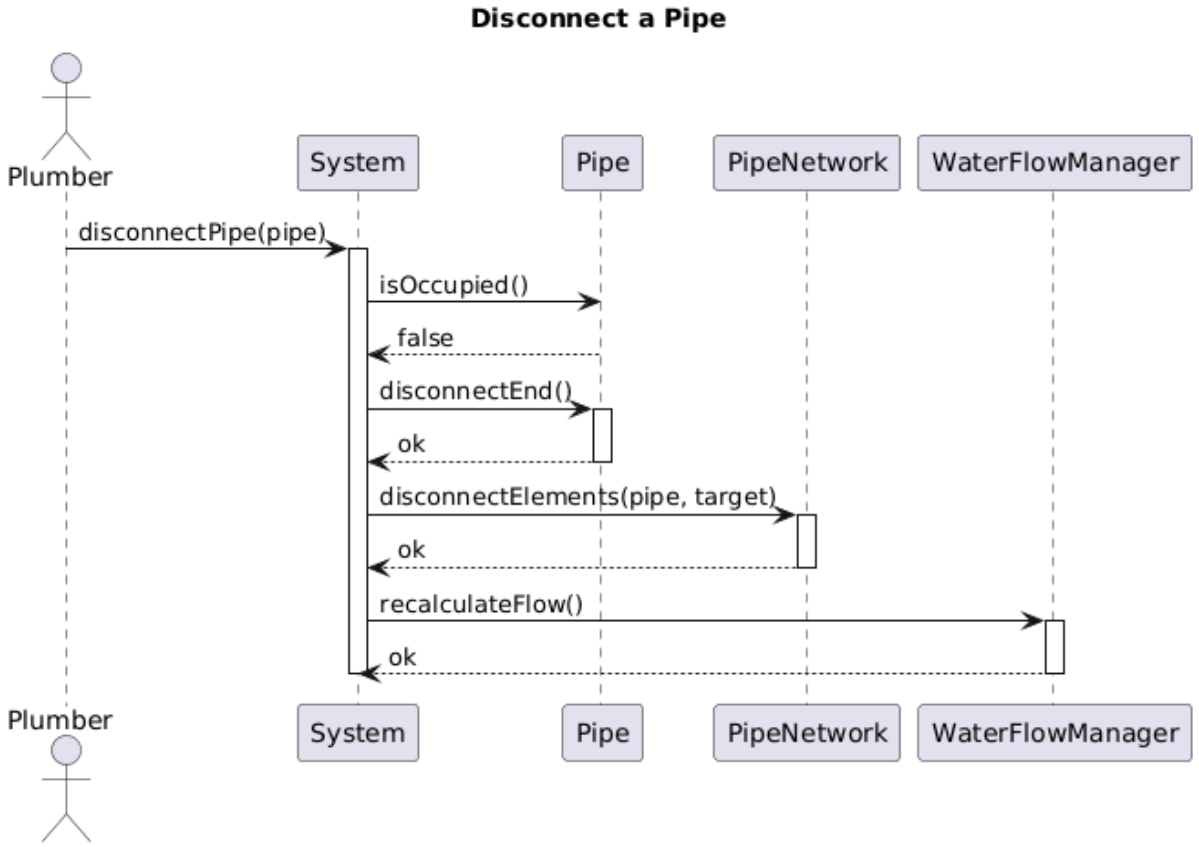
4.4.8 Randomly Break Pump

The System randomly selects a working pump and sets it to a broken state during gameplay. Water flow is recalculated to reflect the pump being out of order.



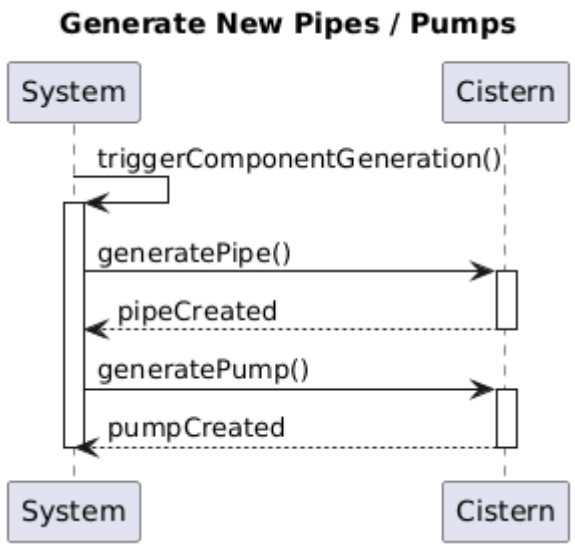
4.4.9 Disconnect a Pipe

A Plumber standing on a pipe disconnects one of its ends from the network. The System verifies the pipe is not occupied, removes the connection, and recalculates water flow. The disconnected end becomes free, meaning any water reaching it will leak to the desert.



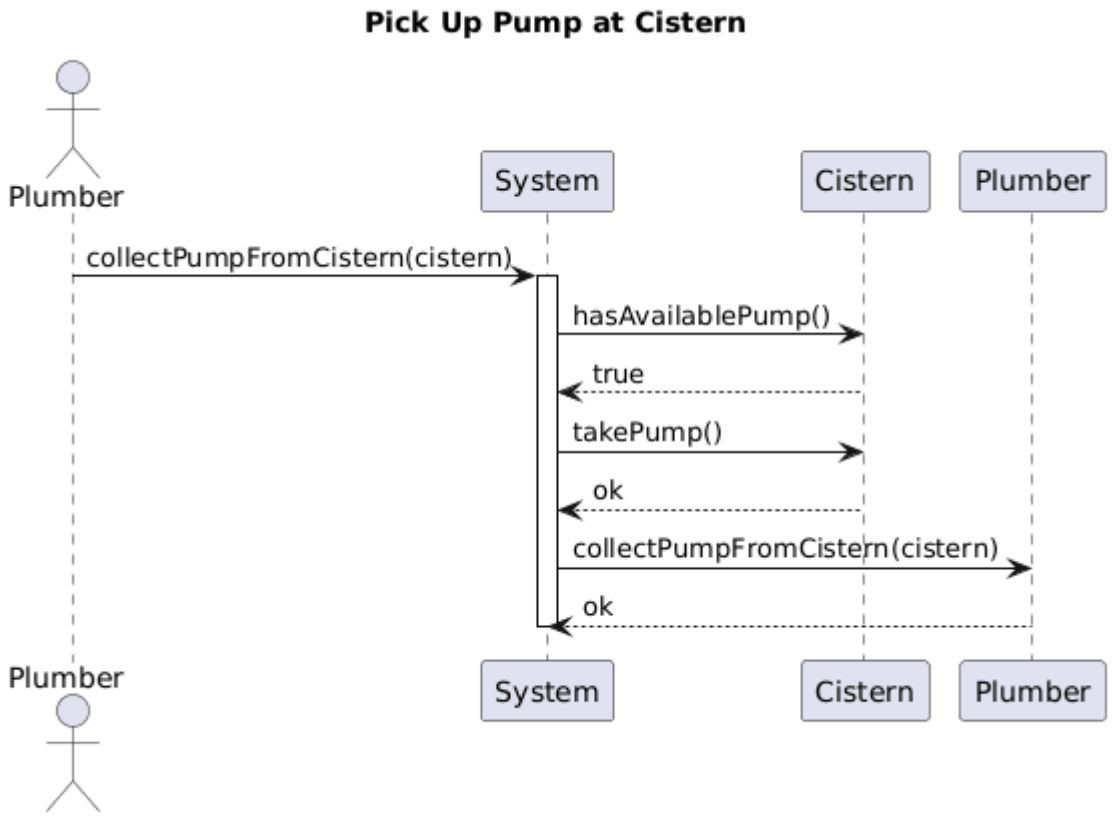
4.4.10 Generate New Pipes / Pumps

The System periodically checks cisterns and generates new pipes or pumps if none are currently available. The new components remain at the cistern until collected by a Plumber.



4.4.11 Pick Up Pump at Cistern

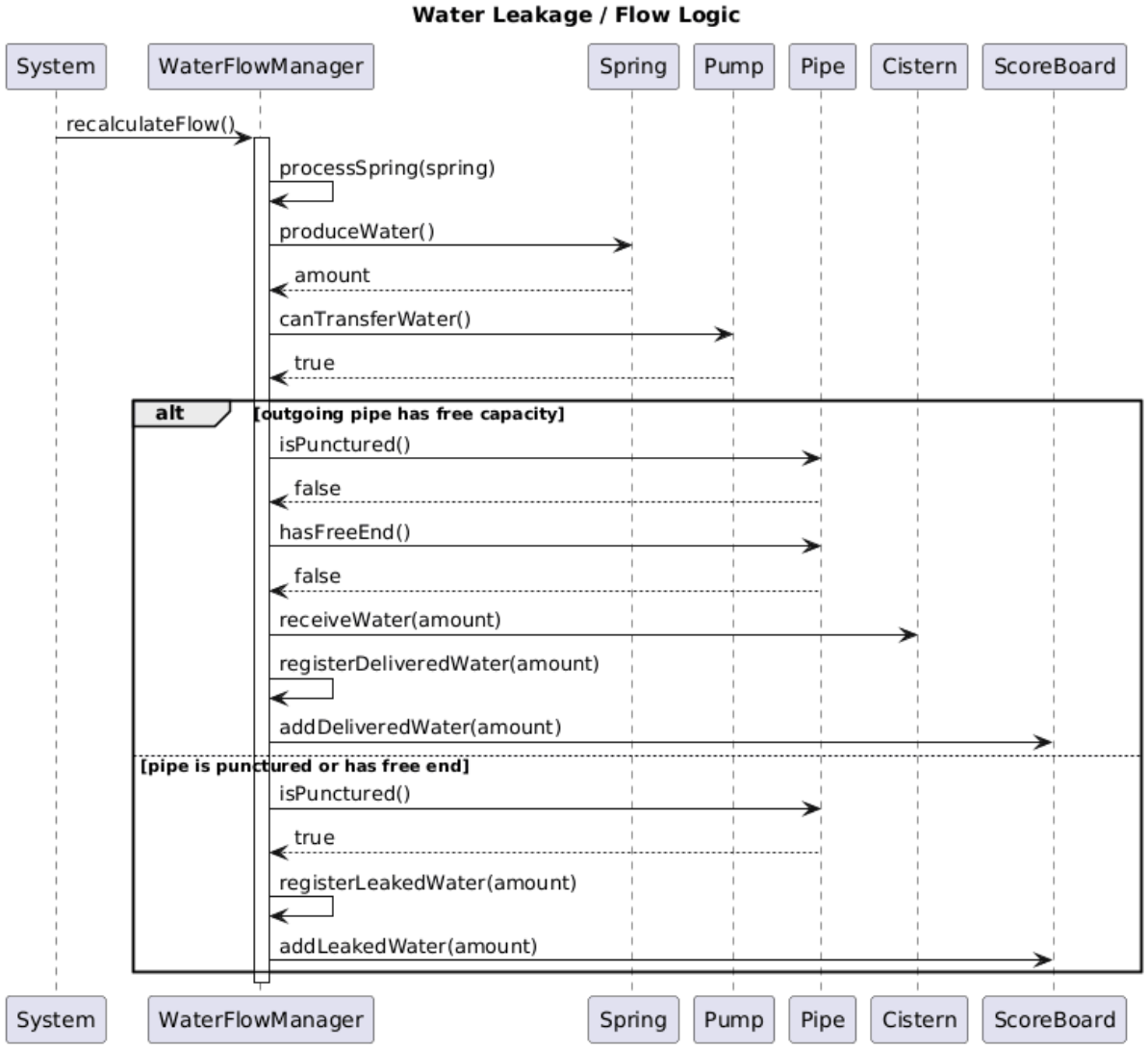
A Plumber standing on a pipe or pump adjacent to a cistern collects water from it. The System checks availability at the cistern, marks the pump as taken, and the Plumber carries it for use in a subsequent turn.



4.4.12 Water Leakage / Flow Logic

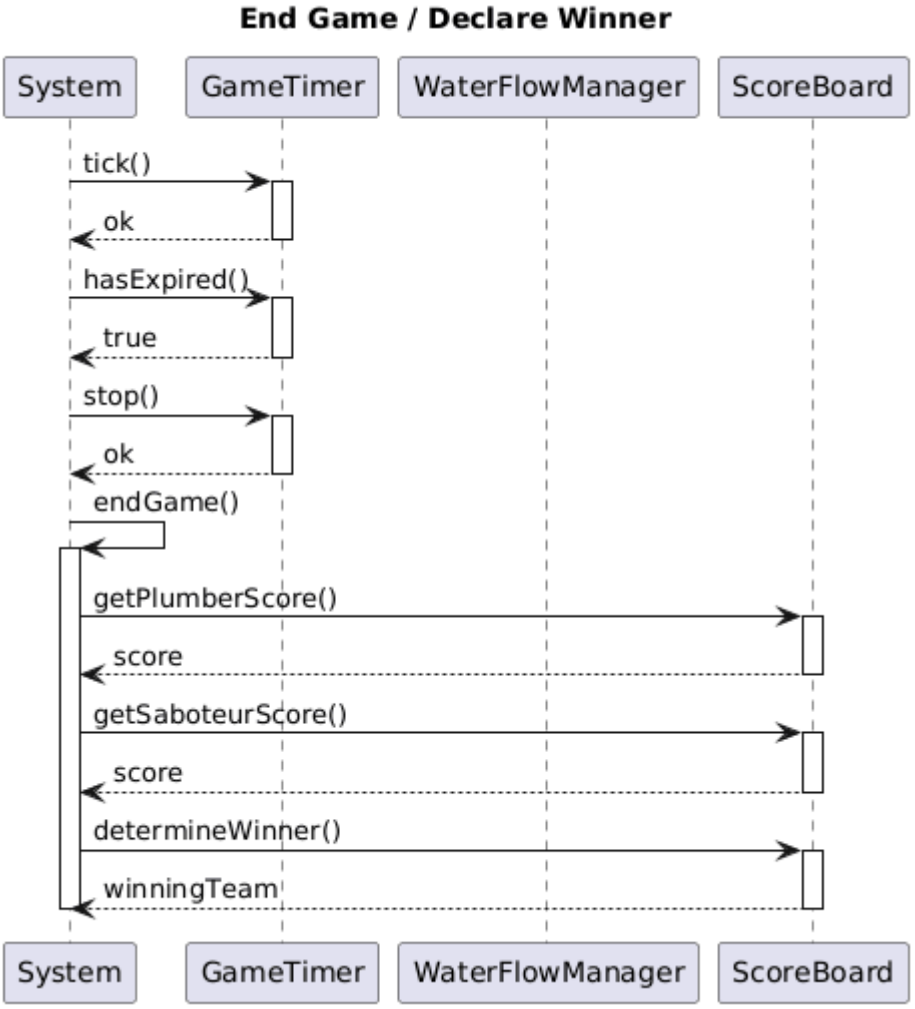
During each game tick the WaterFlowManager processes water movement through the network. A pump transfers its buffered water to the outgoing pipe if capacity allows, then

pulls water from the incoming pipe. Water reaching a punctured pipe or a free pipe end leaks to the desert and is counted as lost, while water reaching a cistern is counted as delivered.



4.4.13 End Game / Declare Winner

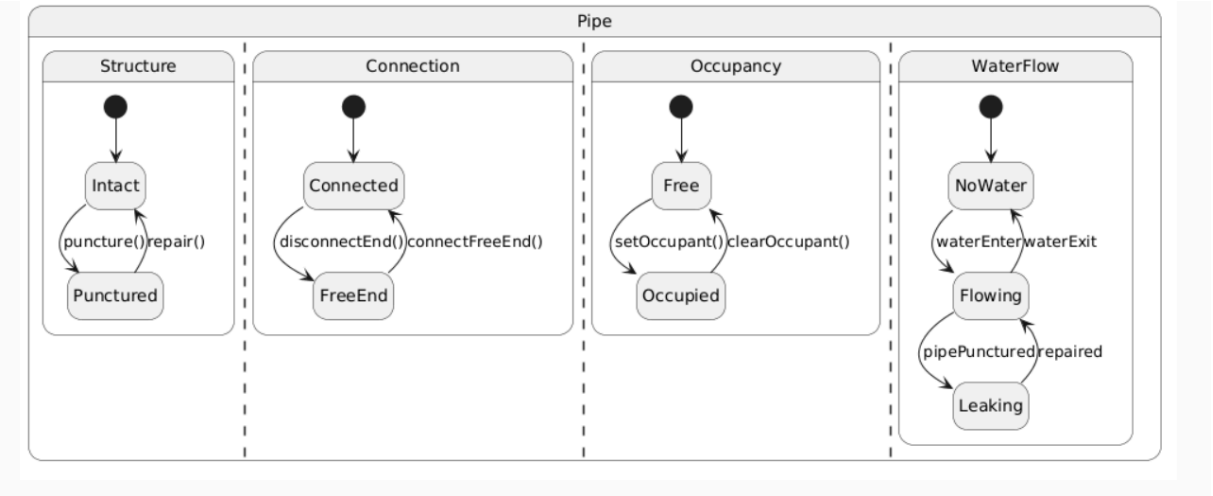
When the game timer expires, the System halts all gameplay and compares the total delivered water against total leaked water using the ScoreBoard. The team with the higher score is declared the winner. Note that the timer-based end condition is a design decision made to resolve the ambiguity in the original problem definition, which only states that the team collecting the most water wins.



4.5 State-charts

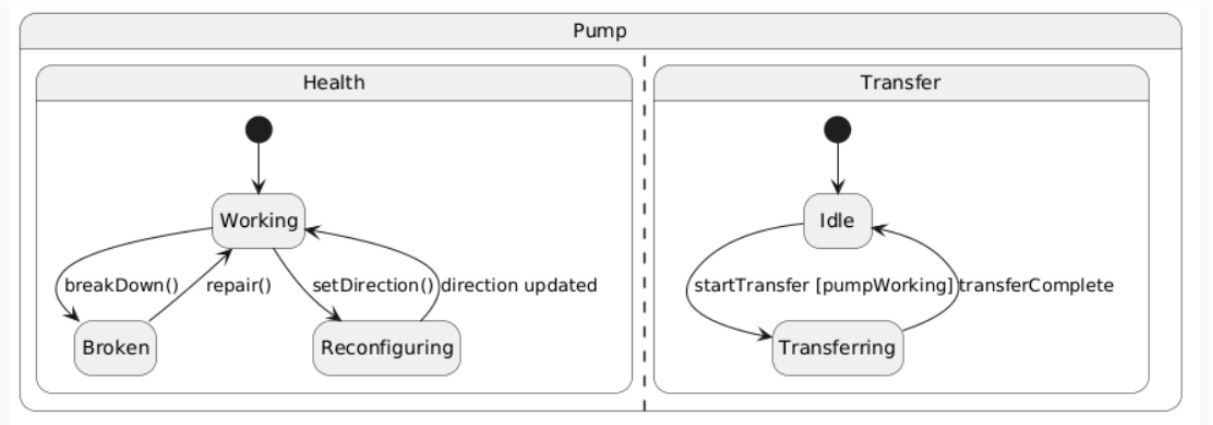
4.5.1 Pipe class

- **Structure:** The pipe can be Intact (no damage) or Punctured (has a hole and leaks water).
- **Connection:** The pipe can be Connected (both ends attached) or have a FreeEnd (one end is free).
- **Occupancy:** The pipe can be Free (no occupant) or Occupied (an element resides inside).
- **WaterFlow:** The pipe can have NoWater, be Flowing (water moves normally) or Leaking (punctured and leaks water).



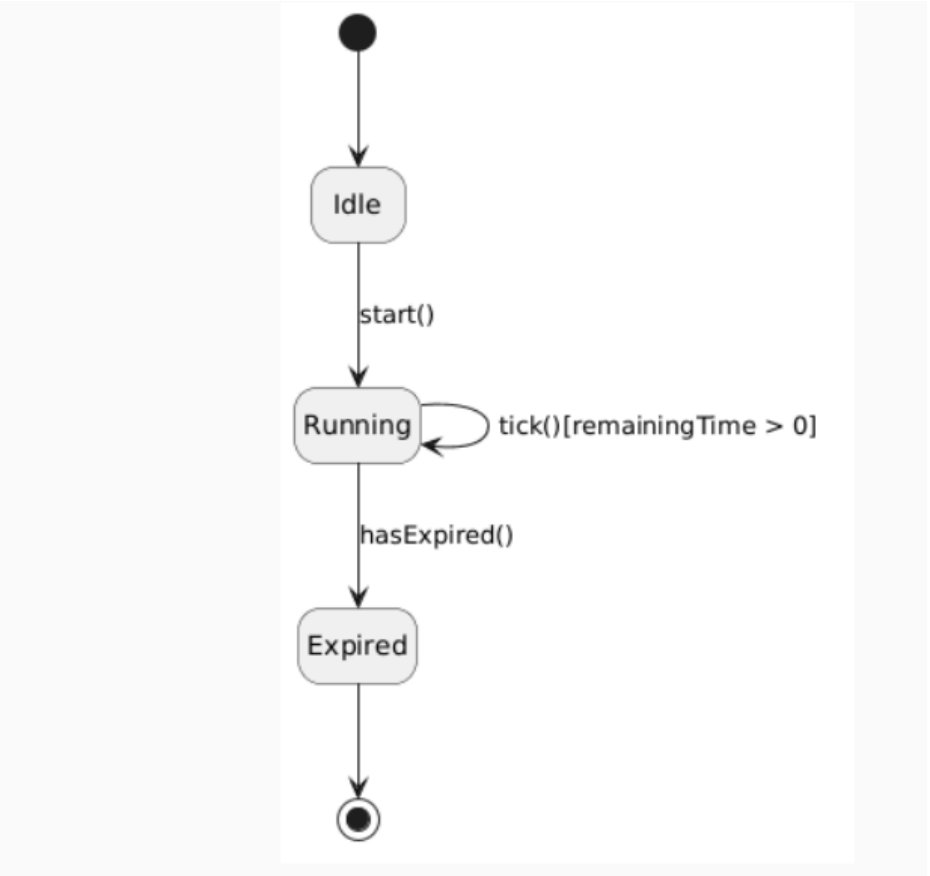
4.5.2 Pump class

- **Health:** The pump can be Working (operating normally), Broken (has broken down) or Reconfiguring (direction is being updated).
- **Transfer:** The pump can be Idle (no transfer in progress) or Transferring (actively moving water).



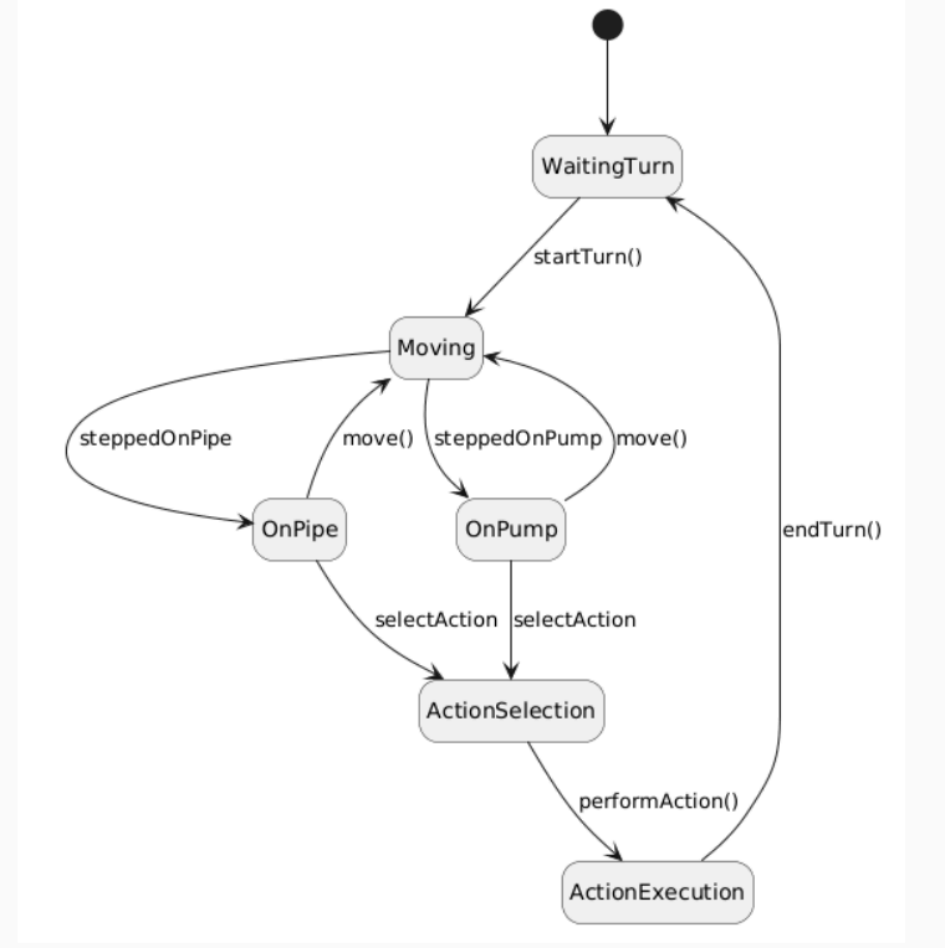
4.5.3 GameTimer class

- **Idle:** The timer has not started yet.
- **Running:** The timer is active and counting down while remainingTime > 0.
- **Expired:** The timer has reached zero and is finished.



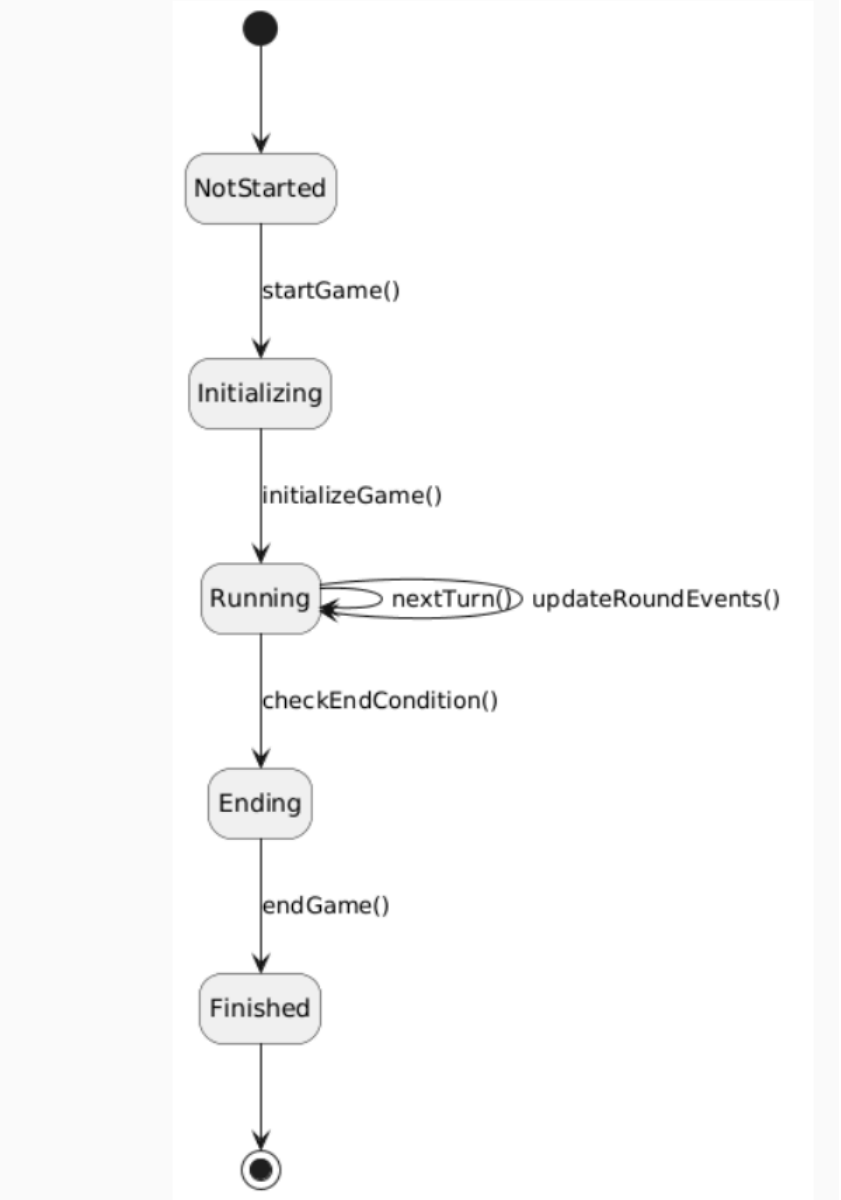
4.5.4 Player class

- **WaitingTurn:** The player is waiting for their turn to begin.
- **Moving:** The player is moving and can land on a pipe or pump.
- **OnPipe:** The player has landed on a pipe and may select an action.
- **OnPump:** The player has landed on a pump and may select an action.
- **ActionSelection:** The player is choosing an action to perform.
- **ActionExecution:** The player is executing the selected action.



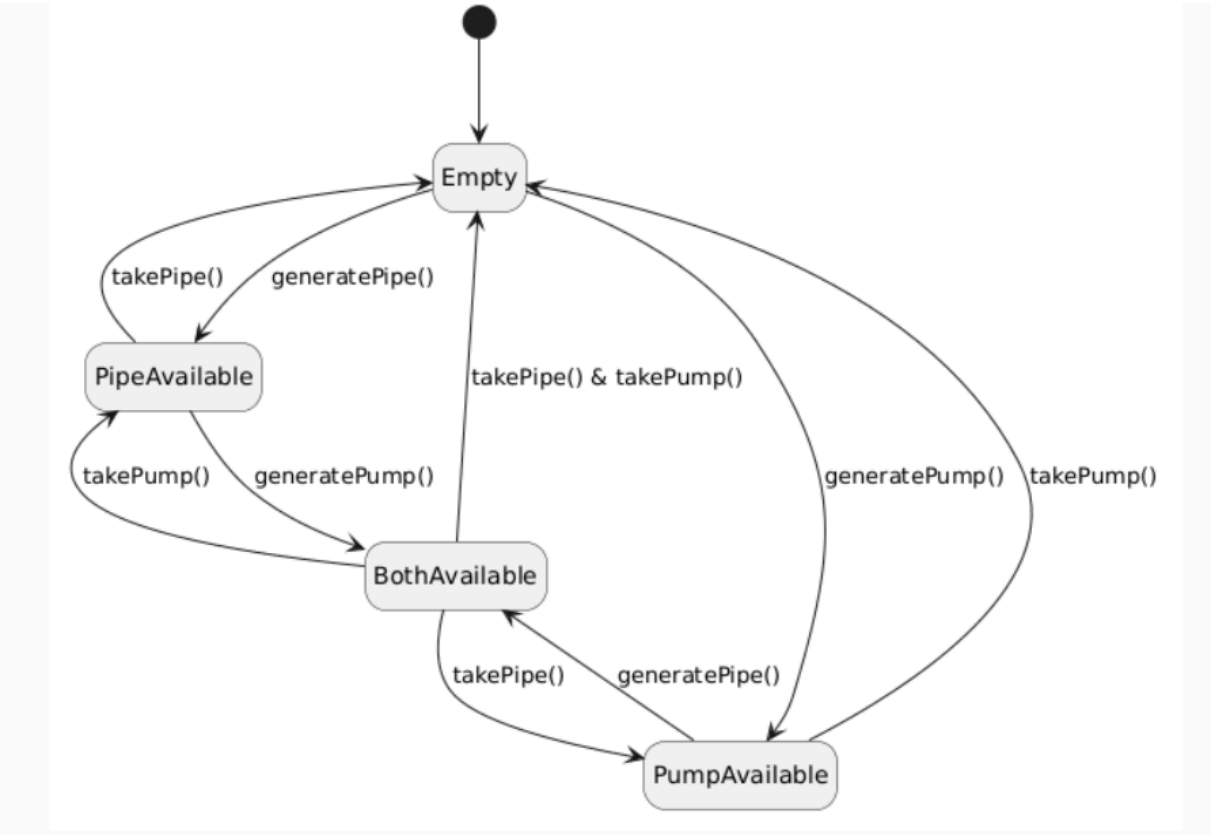
4.5.6 System class

- **NotStarted:** The game has not been started yet.
- **Initializing:** The game is being set up and prepared.
- **Running:** The game is active and progressing through turns.
- **Ending:** The end condition has been met and the game is wrapping up.
- **Finished:** The game is fully complete.



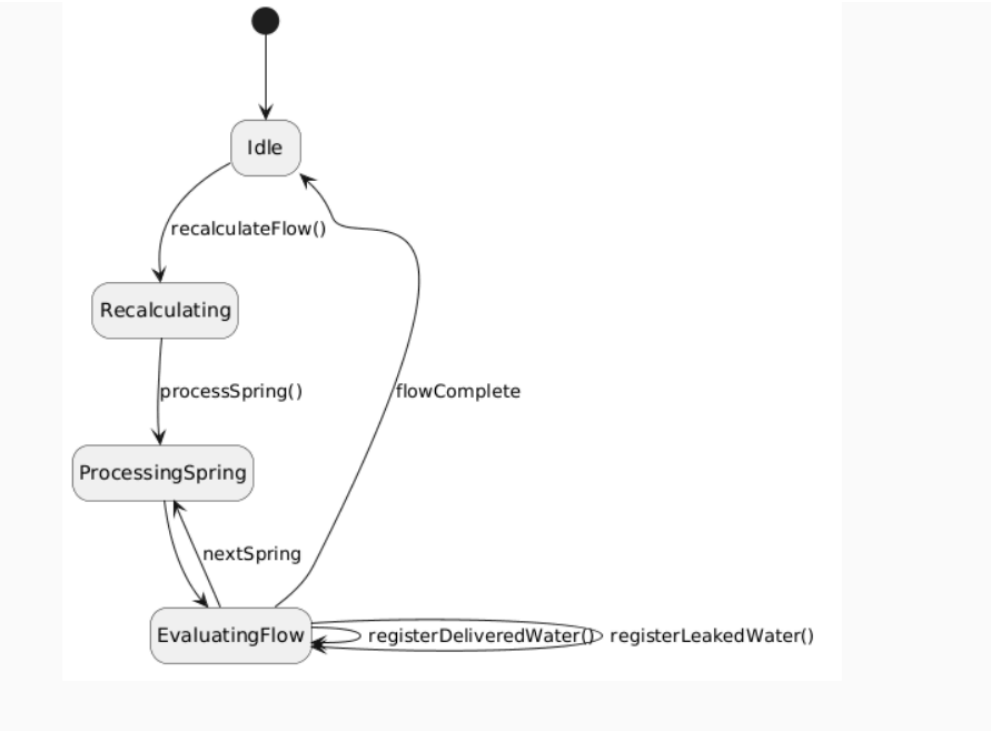
4.5.7 Cistern class

- Empty: The cistern holds no pipes or pumps.
- PipeAvailable: The cistern contains at least one pipe ready to be taken.
- PumpAvailable: The cistern contains at least one pump ready to be taken.
- BothAvailable: The cistern contains both pipes and pumps ready to be taken.



4.5.8 WaterFlow Manager class

- **Idle:** The manager is inactive and waiting for a flow recalculation trigger.
- **Recalculating:** The manager is recomputing the flow distribution across the network.
- **ProcessingSpring:** The manager is processing each spring source one by one.
- **EvaluatingFlow:** The manager is registering delivered and leaked water for the current spring.



4.6 Tools Used

Tool	Purpose	Description
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming ideas, and clarifying edge cases in the documentation.
Gaphor	UML diagram creation	Used to create UML diagrams such as class, sequence, and activity diagrams.
PlantUML	UML diagram creation	Used by some team members to generate UML diagrams in text-based format.
Microsoft Word	Documentation	Used to write, format, and organize the analysis model document.
Discord	Communication	Used for online team meetings and coordination between team members.

4.7 Experiencing AI tools is checking model consistency

4.7.1 Checking if the requirements document is consistent with the problem definition

Conclusion from AI:

The requirements document is largely consistent with the problem definition.

Most system concepts and gameplay rules are represented correctly.

The only notable difference is the introduction of a timer-based game ending, which is not directly stated in the problem definition but is allowed as a reasonable interpretation of an unspecified rule.

Overall, the requirements document correctly reflects the problem definition with only minor assumptions and clarifications added during system design.

4.7.2 Checking UML notation correctness in Requirements and Design documents

We have showed ChatGPT our UML diagrams and asked for its thoughts on our:

- Use Case Diagram
- Static Structure Diagram (Class/Object Model)
- Sequence Diagrams
- State Diagrams

This is the conclusion ChatGPT has given for it:

The UML diagrams in the requirements and design documents follow correct UML notation overall.

No major syntax errors were identified.
The main issues concern model clarity, naming consistency, and level of detail, rather than incorrect UML usage.

Therefore, the UML modeling can be considered technically correct but with room for improvement in readability and behavioral detail.

I don’t agree with the naming consistency is something that AI misunderstood. It confused the names of “Generate New Pipe” with “Insert New Pump” and “Place New Pipe”. Then it thought that we used all of them in the functional requirements while we could have used just one name. These use cases are completely different from each other and are used for different tasks.

4.7.3 Checking if functional requirements have associated use cases

Functional requirements do have associated use cases in the improved model, but this is not fully or cleanly demonstrated in the original requirements document itself. So the correct conclusion is:

Partially yes — but the traceability is incomplete and had to be repaired in the later analysis model.

4.7.4 Checking if use cases are consistent with the sequence diagrams

The use cases and sequence diagrams are mostly consistent. The updated sequence diagrams now represent the main use-case behaviors correctly, with only minor remaining issues in wording, abstraction level, and system-centralized interaction style.

4.7.5 Checking if the static model (object model) has all the concepts of the problem definition

Concept from Problem Definition	Present in Static Model	Result
Pipe	✔	Consistent
Spring	✔	Consistent
Pump	✔	Consistent
Cistern	✔	Consistent
Pipe network	✔	Consistent
Players	✔	Consistent
Teams	✔	Consistent
Water flow	✔	Modeled indirectly by WaterFlowManager
Pump breakdown	✔	Consistent
Pipe leakage	✔	Consistent

4.4.6 Checking if sequence diagrams cover, as a minimum, the situations described in the feedback.

Chatgpt reported that our current sequence diagrams do not yet cover all of the minimum situations from the feedback. It said that "puncturing a pipe, fixing a pipe, setting pump direction, fixing a pump, pump is broken, connecting a pipe, picking up a pipe at a cistern, putting/inserting a pump into a pipe" are missing while current sequence diagrams cover Start

Game, End Game, Move Player / stepping on elements, Disconnect a Pipe, Pick Up Pump at Cistern, and Water Leakage / Flow Logic.

4.7.7 Checking if the static model (object model) and sequence diagrams are consistent (names of objects, attributes, methods…)

The static model and sequence diagrams are largely consistent in terms of object names and most method names.
Inconsistencies:
The strongest issue is in the Move Player sequence diagram, where network-topology operations such as addNeighbor and removeNeighbor are used during player movement, even though these belong to structural connection management rather than movement behavior. In addition, the sequence diagrams show direct interaction between System and WaterFlowManager, but this relationship is not clearly represented in the static model. There is also a minor naming inconsistency in the Cistern attributes between the class description and the class diagram.

4.7.8 Checking if state diagrams are consistent with the rest of the modelling elements

The state diagrams are largely consistent with the other modelling elements in terms of class names and core behaviors. The Pipe, Pump, GameTimer, System, Cistern, and WaterFlowManager statecharts align reasonably well with the corresponding class descriptions and sequence diagrams. The main inconsistency is in the Player statechart. selectAction and performAction that is not defined in the Player class interface
and overlaps with responsibilities assigned elsewhere to Plumber and Saboteur. There is also a minor wording issue in the Pipe statechart. Statecharts are mostly consistent, but the Player statechart should be simplified.

4.7.9 Generate a Requirements Traceability Matrix (RTM) with the AI tool. RTM should contain, as a minimum: Requirement ID, Requirements summary, Requirement in the Problem definition, Use case(s), Main design elements (object / sequence/state).

Requirement ID	Requirement Summary	Problem Definition Reference	Use Case	Main Design Elements (Object / Sequence / State)
FR-01	Enter player names before starting game	Not specified in problem definition	Start / Supervise Game	**Objects:** System, Player, Team **Sequence:** Start / Supervise Game

FR-02	Set game duration	Ambiguous requirement customization	Start / Supervise Game	**Objects:** GameTimer, System **Sequence:** Start / Supervise Game **State:** GameTimer
FR-03	Require at least two plumbers and two saboteurs	"Each team has at least two players."	Start / Supervise Game	**Objects:** Team, Player **Sequence:** Start / Supervise Game
FR-04	Generate an initial network with spring, cistern, pump, pipes	"A complex pipe system transfers water from the mountain springs to the cisterns."	Start / Supervise Game	**Objects:** PipeNetwork, Spring, Pump, Cistern **Sequence:** Start / Supervise Game
FR-05	Start game after conditions satisfied	Derived system rule	Start / Supervise Game	**Objects:** System **Sequence:** Start / Supervise Game **State:** System
FR-06	Display network, players, scores, time	UI representation of system state	Start / Supervise Game	**Objects:** System, ScoreBoard, GameTimer **Sequence:** Start / Supervise Game
FR-07	Water flows from springs through network	"A complex pipe system transfers water from the mountain springs."	Start / Supervise Game	**Objects:**Spring, PipeNetwork, WaterFlowManager **Sequence:** Start / Supervise Game
FR-08	Water reaching cistern counted as delivered	"Plumbers' task is to transfer water… to the cisterns."	Start / Supervise Game	**Objects:** Cistern, ScoreBoard **Sequence:** Start / Supervise Game
FR-09	Water reaching free pipe end counted as lost	"When a pipe has a free end, water flows out to the desert."	Start / Supervise Game	**Objects:** Pipe, WaterFlowManager, ScoreBoard **Sequence:** Start / Supervise Game
FR-10	Pump has one active input and output	"A pump can have only a single input and a single output."	Change Pump Direction	**Objects:** Pump **Sequence:** Change Pump Direction **State:** Pump

FR-11	Water flows only through selected pump route	"Incoming and outgoing pipe can be individually selected."	Change Pump Direction	**Objects:** Pump, WaterFlowManager **Sequence:** Change Pump Direction
FR-12	Broken pump cannot transfer water	"At random times, a pump may go out of order, and water will not flow through it."	Randomly Break Pump	**Objects:** Pump **Sequence:** Randomly Break Pump **State:** Pump
FR-13	Pump stops transfer if output pipe is full	"Pump can transfer water only if the outgoing pipe has some free capacity."	Start / Supervise Game	**Objects:** Pump, Pipe **Sequence:** Start / Supervise Game
FR-14	Pump connection limit enforced	"Number of connectable pipes is specific to each pump."	Insert New Pump / Start Game	**Objects:** Pump, PipeNetwork **Sequence:** Insert New Pump
FR-15	Player moves only along pipes and pumps	"Plumbers and saboteurs can only move on the pipes and the pumps."	Move Player	**Objects:** Player, Pipe, Pump **Sequence:** Move Player
FR-16	Player moves only to adjacent connected element	Pipe system composed of connected elements	Move Player	**Objects:** Player, PipeNetwork, NetworkElement **Sequence:** Move Player
FR-17	Only one player on pipe	"Only a single person can go along a pipe."	Move Player	**Objects:** Pipe, Player **Sequence:** Move Player
FR-18	Multiple players on pump	"People can go around each other at the pumps."	Move Player	**Objects:** Pump, Player **Sequence:** Move Player
FR-19	Turn-based gameplay	Derived game rule	Move Player	**Objects:** System, Player **Sequence:** Move Player **State:** Player

FR-20	One action per turn	Players perform actions on pipe system	Move Player	**Objects:** Player **Sequence:** Move Player **State:** Player
FR-21	Plumber repairs broken pump	"Plumbers can fix broken pumps."	Repair Pipe/Pump	**Objects:** Plumber, Pump **Sequence:** Repair Pipe/Pump **State:** Pump
FR-22	Repaired pump resumes transfer	Pump functionality restored after repair	Repair Pipe/Pump	**Objects:** Pump **Sequence:** Repair Pipe/Pump **State:** Pump
FR-23	Plumber repairs punctured pipe	"Plumbers can fix leaking pipes."	Repair Pipe/Pump	**Objects:** Plumber, Pipe **Sequence:** Repair Pipe/Pump **State:** Pipe
FR-24	Repaired pipe stops leaking	"When a pipe is leaking, water spills out."	Repair Pipe/Pump	**Objects:** Pipe **Sequence:** Repair Pipe/Pump **State:** Pipe
FR-25	Plumber changes pump direction	"Plumbers can set the directions of pumps."	Change Pump Direction	**Objects:** Plumber, Pump **Sequence:** Change Pump Direction **State:** Pump
FR-26	Saboteur changes pump direction	"Saboteurs can change the directions of the pumps."	Change Pump Direction	**Objects:** Saboteur, Pump **Sequence:** Change Pump Direction **State:** Pump
FR-27	Generated pipe at cistern has one free end	"New pipes are manufactured regularly at the cisterns… one end connected, the other free."	Place New Pipe	**Objects:** Cistern, Pipe **Sequence:** Place New Pipe **State:** Pipe
FR-28	Plumber connects free pipe end	"Pipes can be disconnected… and connected to another element."	Place New Pipe	**Objects:** Plumber, PipeNetwork **Sequence:** Place New Pipe **State:** Pipe
FR-29	Plumber collects pump at cistern	"Plumbers can pick up new pumps manufactured at cisterns."	Insert New Pump	**Objects:** Plumber, Cistern, Pump **Sequence:** Insert New Pump

FR-30	Insert pump by splitting pipe	"Pump can be inserted into the middle of a pipe by cutting it."	Insert New Pump	**Objects:** Pump, PipeNetwork, Pipe **Sequence:** Insert New Pump
FR-31	Saboteur punctures pipe	"Saboteurs can puncture pipes."	Puncture Pipe	**Objects:** Saboteur, Pipe **Sequence:** Puncture Pipe **State:** Pipe
FR-32	Punctured pipe leaks water	"When a pipe is leaking, water spills out."	Puncture Pipe	**Objects:** Pipe, WaterFlowManager **Sequence:** Puncture Pipe **State:** Pipe
FR-33	System randomly breaks pumps	"At random times a pump may go out of order."	Randomly Break Pump	**Objects:** System, Pump **Sequence:** Randomly Break Pump **State:** Pump
FR-34	Broken pump remains broken until repaired	Pump stops transferring water until fixed	Randomly Break Pump	**Objects:** Pump **Sequence:** Randomly Break Pump **State:** Pump
FR-35	System generates pipes periodically	"New pipes are manufactured regularly at the cisterns."	Generate New Pipes / Pumps	**Objects:** System, Cistern, Pipe **Sequence:** Generate New Pipes / Pumps **State:** Cistern
FR-36	System generates pumps periodically	"Plumbers can pick up new pumps manufactured at cisterns."	Generate New Pipes / Pumps	**Objects:** System, Cistern, Pump **Sequence:** Generate New Pipes / Pumps **State:** Cistern
FR-37	Game ends when timer expires	"The game is won by the team collecting the most amount of water."	End Game / Declare Winner	**Objects:** GameTimer, System **Sequence:** End Game / Declare Winner **State:** GameTimer
FR-38	Compare delivered vs lost water	"Saboteur team leaks water; plumber team transfers water."	End Game / Declare Winner	**Objects:** ScoreBoard, WaterFlowManager **Sequence:** End Game / Declare Winner
FR-39	Winner determined from scores	"The game is won by the team collecting the	End Game / Declare Winner	**Objects:** ScoreBoard, Team

		most amount of water.”		**Sequence:** End Game / Declare Winner
FR-40	Display final winner	“Game is won by the team collecting the most amount of water.”	End Game / Declare Winner	**Objects:** System, ScoreBoard **Sequence:** End Game / Declare Winner **State:** System
FR-41	The game session runs for a predefined duration controlled by a timer. When the timer expires, the system compares the water delivered to cisterns with the water lost to the desert and determines the winning team.	“The game is won by the team collecting the most amount of water.” (the problem definition does not specify when the game ends)	End Game / Declare Winner	**Objects:** GameTimer, System, ScoreBoard **Sequence:** End Game / Declare Winner **State:** GameTimer, System

4.7.10 Point out the cases when bidirectional traceability is not provable.

Bidirectional traceability is not fully provable in the current submission for 3 main reasons:

- First, several functional requirements are grouped under broad use cases such as *Start / Supervise Game* and *Move Player*, so reverse mapping from a use case or sequence diagram to one exact requirement is not one-to-one.
- Second, version 2 does not include explicit sequence diagrams for several required actions such as *Puncture Pipe*, *Repair Pipe/Pump*, *Change Pump Direction*, *Insert New Pump*, *Randomly Break Pump*, and *Generate New Pipes / Pumps*, therefore backward traceability from those requirements/use cases to sequence diagrams cannot be demonstrated.
- Third, some problem-definition concepts, especially *water* and *pipe end*, are represented only indirectly rather than as explicit objects, so traceability from the original problem definition to concrete model elements is weaker in those cases. Finally, the timer-based end condition is a design interpretation and therefore not a direct backward trace to the original problem text.

4.8 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
2026.03.10 18:00	2 hours	Arda Ibrahim Ilgın Hameez Yahya Aasif	Online team meeting conducted on Discord to review the professor's feedback on the submitted Analysis Model. The team discussed the remarks related to the static structure diagram, sequence diagrams, and state charts. The feedback was analyzed carefully and the main improvement points were identified, including revisiting relationships between pipes, pumps, cisterns, springs, and players, improving readability of the static structure diagram, and extending the behavioral diagrams to better represent system actions. During the meeting the team agreed on how to update the diagrams according to the professor's remarks and the original problem description. Tasks for the next working session were delegated among members: Ilgın and Arda were assigned responsibility for revising and improving the class diagram and checking relationships between domain objects; Ibrahim and Yahya were responsible for reviewing and updating the sequence diagrams to ensure that all required interactions are represented; Hameez and Aasif were assigned to revise and expand the state charts to better reflect system states and transitions. The team agreed that the corrections would be completed during the next in-person working session.
2026.03.10 09:00	9 hours	Arda Ibrahim Ilgın Hameez Yahya Aasif	Full team working session conducted physically at the **Corvinus University Library.** The objective of the meeting was to update and refine the Analysis Model according to the professor's remarks. At the beginning of the session the team reviewed the previously discussed feedback and confirmed the task delegation decided during the earlier Discord meeting. The work was then divided among three groups. Ilgın and Arda worked on revising and reorganizing the class diagram, improving its readability, checking associations between

			classes, and ensuring that the relationships between pipes, pumps, cisterns, springs, and players are correctly represented. Ibrahim and Yahya focused on reviewing and refining the sequence diagrams, verifying that each use case is properly represented and that system interactions match the functional requirements and use-case descriptions. Hameez and Aasif worked on improving the state diagrams by expanding the states and transitions so that they better reflect the behaviors of the main system components. Throughout the session the team regularly reviewed each other's work to ensure consistency between the diagrams, the requirements document, and the original problem definition. Decisions were made regarding diagram structure, naming conventions, and relationships between system elements in order to comply with the professor's comments. At the end of the session all diagrams were reviewed collectively, final corrections were applied, and the updated documentation was finalized for submission.
2026.03.12 17:00	4 hours	Ibrahim Arda Ilgın	In the final documentation session, each member worked individually and contacted via WhatsApp group chat to share the information and thoughts. Arda prepared section 4.7 Experiencing AI Tools, where AI was used to check model consistency across requirements, use cases, and design elements. Ilgın generated the Requirements Traceability Matrix (RTM) using an AI tool, linking requirement IDs to the problem definition, related use cases, and main design elements (objects, sequence diagrams, and state diagrams). Ibrahim added the missing sequence diagrams, prepared section 4.6 Tools Used, and finalized the document formatting and consistency check before submission.